SCULPTOR Has Your Back(up Path): Carving Interdomain Routes to Services

Paper #559, 12 pages body, 19 pages total

Abstract

Cloud and content providers (Service Providers) serve diverse Internet applications — enterprise services, gaming, video, real-time AI inference — with goals that pull in different directions: low latency for some flows, high availability for others, low cost for bulk transfers, and graceful behavior when peering links fail or flash crowds arrive. To meet these goals Service Providers either overprovision, react too late, or overdesign specific solutions. We take a different approach with SCULPTOR, a framework that proactively optimizes BGP configurations to satisfy diverse ingress routing objectives-including latency, cost, and capacity constraints-simultaneously. SCULPTOR models Internet routing to approximately solve a large integer optimization problem at scale using gradient descent. We prototyped SCULPTOR on a global public cloud and tested it in real Internet conditions, demonstrating that it reduces traffic overloaded during site failures by up to 40%, achieves optimal max-link-utilization, routes high-priority traffic with 50% less overloading, all while staying within 1.3 ms of an optimal that requires 19× more prefixes..

1 Introduction

Cloud/content providers (hereafter Service Providers) enable diverse Internet applications used daily by billions of users. Traditionally, Service Providers used DNS and static BGP advertisements to define a single path from a user to a service, leaving the specific path up to the Internet to determine.

Increasingly, however, these services have diverse requirements that can be challenging to meet with a single path that Service Providers have little control over. For example, enterprise services have tight reliability requirements [1, 2], and new applications such as virtual reality require ≤ 10 ms round trip latency [3] and ≤ 3 ms jitter [4]. Complicating matters, Service Providers must meet these requirements subject to changing conditions such as peering link/site failures [5, 6] and DDoS attacks [7, 8, 9]. Critically, such changes can cause *overload* if the Service Provider cannot handle new traffic volumes induced by the change. This overload can lead to degraded service for users, hurting reliability [10, 11, 12, 2], and may require manual intervention.

To try to meet different goals and respond to changing conditions, Service Providers adopt solutions that are either (a) too costly, (b) too reactive, or (c) too specific. For example, Service Providers take a *costly* approach by proactively overprovisioning resources [13, 14, 15], but we demonstrate using traces that bursty traffic patterns can require overprovisioning rates as high as 70% despite sufficient global capacity (§2.3). TIPSY *reactively* responds to changing loads to retain reliability \cite{tipsy, but reactivity could lead to short-term degraded performance. AnyOpt and PAINTER proactively set up routes to optimize *specific* objectives such as steady-state latency [16, 1], but it is challenging to extend those approaches to other objectives (§5.4.1).

We present Service Providers with a flexible framework, SCULPTOR (**S**couring **C**onfigurations for **U**tilization-**L**oss-and **P**erformance-aware **T**raffic **O**ptimization & **R**outing), which accepts as input general objectives and outputs BGP advertisements and traffic allocations that help achieve those desired objectives. SCULPTOR is the first system that finds good solutions for ingress interdomain routing objectives such as maximum link utilization, transit cost, and latency for *interdomain* traffic (existing systems work for intradomain traffic, *e.g.,* [17, 18, 19, 20]). SCULPTOR computes BGP advertisements proactively without affecting live traffic, only placing live traffic on new routes after convergence, mitigating risk (§4).

To solve each traffic engineering problem, SCULPTOR efficiently searches over the large BGP advertisement search space (> $2^{10,000}$ possibilities, *i.e.,* from > 10,000 peerings) by modeling how different strategies perform, without having to predict the vast majority of actual paths taken under different configurations since predicting interdomain paths is hard and measuring them on the Internet is slow (§3.4). SCULPTOR then finds configurations using gradient descent: descent (as opposed to greedy/random searches in prior work) enables SCULPTOR to assess > 1M configurations per measurement on the Internet (§5.4.1), 1,000× more than other solutions. Gradients helps us smooth over noisy objective function (*i.e.,* routing) estimates (§3.5). Like other work that uses gradient descent with success (*e.g.,* deep learning), we sacrifice the ability to provide a formal characterization of which objective functions are possible for an approach that lets us approximately optimize multiple criteria (§5)

We prototype and evaluate our framework at Internet scale using the PEERING testbed [21] (§4), which is deployed at 32 Vultr cloud locations [22]. Vultr is a global public cloud that allows our prototype to issue BGP advertisements via more than 10,000 peerings. We demonstrate `SCULPTOR`'s utility across many practical objectives: minimizing latency and max-link-utilization, optimizing performance in dynamic traffic conditions, routing latency-sensitive and bulk traffic classes, directing traffic to lower-cost sites, and enabling highly latency-sensitive services. We compare `SCULPTOR`'s performance on these problems to that of an unreasonably expensive "optimal" solution (computing the actual optimal is infeasible). We make an anonymized implementation of `SCULPTOR` available at https://anonymous.4open.science/r/sculptor_stripped-FE7C and will release all code upon acceptance.

On the Internet, measured from RIPE Atlas probes, `SCULPTOR` cuts latency relative to the best prior approach by 3.6 ms in steady state, 6.3 ms during a link failure, and 12 ms during a site failure, failures under which `PAINTER` overloads **TBD:internet.linkfail.overloaded.painte**% and 95.0% of traffic respectively (§5.2).

In emulations seeded by measurements from tens of thousands of networks (§5.3), `SCULPTOR` is the best deployable solution on every objective that we evaluate. It achieves optimal max-link-utilization, achieves steady-state latency within 1.3 ms of optimal. `SCULPTOR` also achieves 64% lower latency during link failures, overloads 40% less traffic than under the best prior approach (`Static Traffic Failover`), leaves 22% less traffic beyond 10 ms of its optimal latency, carries 1.6× the bulk traffic before high-priority traffic congests, and lowers traffic site cost by 72% compared to the best prior approaches.

After decades of intradomain traffic engineering using programmable networking primitives such as virtual output queueing for differentiated service [23] (and others, §7), `SCULPTOR` brings such control closer to realization in the interdomain setting with a unilaterally deployable framework. Service Providers can use flexible frameworks such as `SCULPTOR` to help bring us the resilient, performant service that our diverse applications increasingly need.

2 Motivation

2.1 Variable, Evolving Goals

Evolving Internet use cases are pushing Service Providers to meet diverse requirements. For example, Service Providers increasingly host mission-critical services such as enterprise solutions [1], which are predicted to be a $60B industry by 2027 [24]. These enterprise solutions require high reliability, whereas low latency is a more important metric for gaming [3], which generates more revenue than the music and movie industries combined [25]. Service Providers are also expanding the set of services they provide—for example, CDNs that traditionally hosted static content are pivoting to offering services like compute [26, 27, 28].

Moreover, as use cases evolve, Service Providers increasingly need to meet performance requirements for *ingress* traffic since that traffic includes, for example, player movements in real-time games, voice and video in enterprise conferences, and video/image uploads for AI processing in the cloud [29, 30]. This reality is a departure from traditional CDN traffic patterns in which ingress traffic was primarily small requests and TCP acknowledgments.

Despite efforts to meet variable service requirements [6, 31, 32, 33, 34, 1, 15], doing so is still challenging due to issues *outside* Service Provider control.

First, Service Providers lack control over which ingress path traffic takes because the source and upstream networks determine the route. BGP computes paths in a distributed fashion where each AS selects the single best path to the destination and propagates it; Service Providers cannot force a source network to select a different path if the source prefers an alternative based on its own local preference. Service Providers cannot even unilaterally control the part of the path closest to them—upgrading peering capacity requires coordination among multiple parties [35].

Second, dynamic factors outside Service Provider control make satisfying requirements even harder. Peering disputes can lead to congestion on interdomain links and inflated paths [5, 36, 37], and DDoS attacks still bring down sites/services [9, 10, 11, 12], despite considerable effort in mitigating attacks [7, 8]. Moreover, recent work [38, 39, 20] and blog posts [40, 41, 42, 43, 44, 45] show that user traffic demands are highly variable due to flash crowds and path changes and so can be hard to plan for.

2.2 Standard Approaches

To meet evolving goals, today most Service Providers either use `anycast` prefix advertisements to provide low latency and high availability at the expense of some control [46, 47, 48, 49], or `unicast` prefix advertisements to direct users to specific sites [50, 51, 35, 52]. `Anycast`, where Service Providers advertise a single prefix to all peers/providers at all/many sites, offers some natural availability following failures, since BGP automatically reroutes most traffic to avoid the failure after tens of seconds [48]. Prior work shows that this availability comes with higher latency in some cases [53, 47]. `Unicast` gives clients lower latency than `anycast` by advertising a unique prefix at each site [50, 52], but can

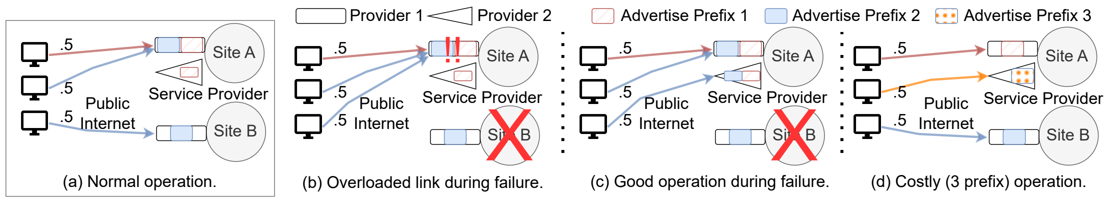


Figure 1: In normal operation traffic is split between two sites by directing half the traffic to each prefix using DNS (a). Even though there is enough global capacity to serve all traffic when site B fails, there is no way to split traffic across multiple providers given available paths, leading to overload (b). A proactive solution is to advertise prefix 2 to an additional provider at site A *a priori*, allowing traffic splitting across the two links (c). A simpler but prohibitively expensive solution is to proactively advertise one prefix per peering (d).

suffer from reliability concerns, taking as much as an hour to shift traffic away from bad routes due to stale DNS records [1].

2.3 New Systems & Their Limitations

In response to some of the basic limitations of `anycast` and `unicast`, others have proposed specialized systems. However, these solutions come with three key limitations.

Too Specific: Some have noted that, by intelligently advertising different prefixes to subsets of peers/providers to offer users more paths, Service Providers can achieve better performance [5, 46, 16, 49, 1]. However, these solutions rely on objective-specific solution methods that do not easily generalize to multi-constraint optimization problems (**??**).

Figure 1 shows how this specific focus on a single objective could lead to reliability issues, *e.g.,* during a site failure. In normal operation, user traffic is split evenly across two prefixes and achieves low latency (Fig. 1a). However when site B fails, BGP chooses the route through Provider 1 for all traffic, causing link overutilization and subsequent poor performance for all users (Fig. 1b). In Section 5 we show that this overload happens in practice for state-of-the-art systems that provide low (steady-state) latency from users to Service Provider networks such as `AnyOpt` [16] and `PAINTER` [1].

Too Reactive: Systems like `TIPSY` [6] enhance reliability by withdrawing BGP announcements to drain overloaded links. However, changing the set of paths is known to lead to outages [54, 55, 56, 15], and can take time to converge. Ideally, we should modify announcements for live traffic only as a last resort.

Too Costly: Service Providers often overprovision to handle transient peak loads [13, 14, 15], but this approach can incur high costs as Figure 2 demonstrates using link utilization data (per 5 minutes) from OVH cloud [57].

To generate Figure 2 we first split the dataset into successive, non-overlapping 120-day planning periods and compute the 95^{th} percentile ingress link utilization ("near-peak load") for each link/period. We then assign each link's capacity for the next period as the near-peak load in the current period multiplied by an overprovisioning factor (X-axis variable). We then compute the average link utilization in the next period and the total number of links on which we see ≥ 100% utilization in at least one 5-minute interval.

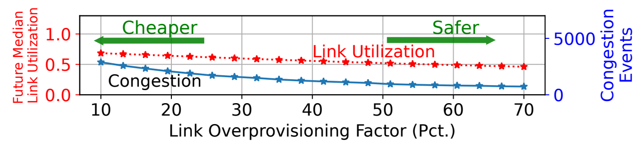


Figure 2: Planning for peak loads to avoid overloading requires inefficient overprovisioning.

Figure 2 plots the median utilization across links and periods and the number of congestive events as we vary the overprovisioning factor. Figure 2 shows that overprovisioning to accommodate peak loads introduces a tradeoff between inefficient utilization and overloading. Low overprovisioning factors between 10% and 30% lead to more efficient utilization (60%-70%) but lead to thousands of congestive events. High overprovisioning factors lead to far less overloading, but only 50% utilization. These events span 41 OVH sites and 1,650 links, indicating a widespread challenge. Results were consistent with larger planning periods, while smaller periods increased congestion.

An important note on cost.: We model deployment cost as correlated with the peering capacity over all links/sites, even though the actual relationship may be complex and additional peering capacity itself is not prohibitively expensive. Upgrading capacity has fixed costs (*e.g.,* fiber, router backplane bandwidth, line card upgrades, CDN servers near peering routers) and variable costs (*e.g.,* power, transit), both of which scale with demand [58, 14, 20, 59]. Hence the high "Link Overprovisioning Factor" to ensure low congestion should be extrapolated out to the general "cost" of upgrading Service Provider serving infrastructure, rather than just a peering link.

Efficient, Proactive Planning is Possible: Despite these limitations, Figure 1c shows that by advertising prefix 2 to provider 2 at site A a priori, the Service Provider can split traffic between the two links during failure, avoiding overloading without (a) reacting to the failure and (b) upgrading deployment capacity.

2.4 Key Challenges

Since a Service Provider has lots of global capacity, dynamically placing traffic on paths to satisfy performance objectives subject to capacity constraints would therefore be simple if all the paths to the Service Provider were always available to all users as in Figure 1d. However, making paths available uses IPv4 prefixes, which are expensive and contribute to the global routing table's memory footprint. The solution in Figure 1d uses 50% more prefixes than the solution in Figure 1c and, at $20k per prefix for 10,000 peerings would cost $200M [60]. IPv6 is not a good alternative as it is not supported by 60% of users according to APNIC [61]. Moreover, adoption is slowing, with adoption in the US stagnating at 50% for a few years, and full adoption possibly taking until 2045 according to APNIC [62] and Google [63]. Prior work noted the same but found that advertising around 50 prefixes was acceptable [16, 1], and most Service Providers advertise fewer than 50 today according to RIPE RIS BGP data [64]. We also verified with engineers at six Service Providers that this challenge is important to address.

Since we cannot expose all the paths by advertising a unique prefix to each connected network, we must find some subset of paths to expose. Finding that right subset of paths to expose that satisfies performance objectives, however, is hard since there are exponentially many subsets to consider, and each subset currently needs to be tested (*i.e.,* advertised via BGP) to see how it performs. The number of subsets is on the order of 2 to the number of ingresses Service Providers have, which, for some Service Providers (including Vultr, which we measure) [65], is $> 2^{10,000}$. As measuring this many advertisements is intractable, we have to predict how different subsets of paths perform, which is challenging since interdomain routing is difficult to model [66, 6].

3 Methodology

3.1 Overview

SCULPTOR takes as input a Service Provider's objective, its prefix budget, and a total training time (§3.2.2). (It also accepts required inputs to compute the objective, *e.g.,* traffic volumes and user latencies.) It outputs an advertisement configuration that gives users good interdomain paths relative to that objective, and an allocation of traffic to those paths. SCULPTOR computes BGP advertisements proactively, only placing live traffic on them after computation (§4).

To search for good advertisements and traffic allocations, SCULPTOR solves two key challenges. First, SCULPTOR cannot easily compute the objective. The only way to know exactly how users will route to a configuration is to advertise it, which can be done at most a few times per hour and so cannot be done for the millions of configurations that an optimization considers. SCULPTOR instead maintains a probabilistic (*i.e.,* noisy) model of how users route, uses it to estimate the objective, and refines it with occasional measurements (§3.4).

Second, the configuration space is large and discrete. SCULPTOR searches it using gradient descent over a relaxed, real-valued configuration which helps smooth the convergence process under a noisy objective (§3.5). Each gradient entry is computed by solving a small traffic-allocation problem (§3.6).

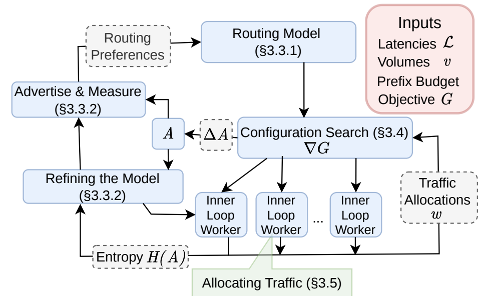


Figure 3: SCULPTOR advertisement computation overview.

Figure 3 shows how these pieces fit together, and in Section 5.4.1 we demonstrate that each piece is important for SCULPTOR's success. We also outline a full iteration of SCULPTOR in Section A.2.

3.2 Problem Background

3.2.1 Setting and Routing Model

Service Providers offer their services from tens to hundreds of geo-distributed sites [67]. The sites for a particular service can serve any user, but users benefit from reaching a low-latency site for performance. Sites consist of sets of servers that have an aggregate capacity. Service Providers also connect to other networks at sites via dedicated links or shared IXP fabrics that we call peering links. Each such link also has a capacity. When utilization of a site or link nears/exceeds the capacity, performance suffers, so Service Providers strive to avoid high utilization [5, 20]. Resources can also fail due to, for example, physical failure and misconfiguration.

Users route to the deployment through the public Internet to a prefix over one of the peering links via which that prefix is advertised. The path to a prefix (therefore peering link) is chosen via BGP. We model interdomain paths from users to the Service Provider as non-overlapping except at the peering link through which each path ingresses to the deployment. We identify a path by its ingress link rather than its neighbor AS, because one AS may peer at many sites.

We assume that the peering link is the bottleneck of each path, for two reasons. First, this link increasingly represents the most important part of the path for Service Providers due to Internet flattening [68]. Second, other parts of the path are less interesting for traffic engineering—last-mile bottlenecks are common to all paths for a user, and existing systems perform traffic engineering in intermediate networks. Handling unexpected bottlenecks (*i.e.,* in a peering network) can also be viewed as a path failure, which we evaluate in Section 5.3.2, *demonstrating that SCULPTOR handles violations of this assumption.*

We assume the Service Provider has some way to direct each UG's traffic among its prefixes, such as DNS [50, 52, 46, 51], multipath transport [69, 70, 71], or control points near users [72, 1]. These options differ in how deployable and how precise they are as PAINTER discusses [1]. SCULPTOR makes no particular assumption about which is used.

3.2.2 Inputs

The high-level input is the Service Provider's *goal* which they express via an objective function, G. The Service Provider hopes to accomplish this goal via a *prefix budget* which is generally much less than the number of total ingresses, and a maximum *training time* or, equivalently, a maximum number of BGP measurements, since SCULPTOR is bottlenecked by BGP measurement time (§3.4).

Oftentimes, objectives are functions of how "well" users route to the deployment, so SCULPTOR also accepts information about users. We aggregate users into user groups (UG*s*), for example /24s, metro-AS pairs, or ASes. The granularity is chosen to be one at which all users in a UG route to the Service Provider the same way, so that a UG has one route per prefix (§3.3). We use /24s (§4).

UG*s* generate steady-state traffic volumes, $v(\texttt{UG})$, and the Service Provider provisions capacity at links/sites to accommodate this load. A system run by the Service Provider measures latency $\mathcal{L}$ and traffic volumes from UG*s* to Service Provider peering links l with known capacities $c(l)$, which are also known inputs in prior work [73, 52, 74, 75, 16, 1]. Part of these latency measurements includes measuring which peering links are *reachable* (in a policy-compliant routing sense) from each UG. SCULPTOR is resilient to changes in latency and traffic over time (§5.3.2).

3.3 The Key Goal

Finding advertisement configurations that admit good assignments of user traffic to paths is a multivariate optimization problem over both configurations and traffic assignments. Section A.1 tabulates variables and expressions.

We represent an advertisement configuration A as a binary vector. Each entry indicates whether we advertise/do not advertise a particular prefix to a particular peer/provider at a particular site (similar to prior work [16, 1]). Implementing this configuration (*i.e.,* advertising prefixes via BGP sessions) results in routes from UG*s* to the Service Provider. A UG has one route per prefix but a UG may have their traffic fractionally assigned to different prefixes. The assignment of traffic to resulting routes from this configuration is given by the non-negative real-valued vector w, whose entries specify traffic allocation for each UG along each route (equivalently, prefix).

Announcing a configuration will result in some set of routes (although knowing exactly which routes a priori is challenging), and these routes define which ingress link a UG uses for a prefix. We can think of this process as a routing function R that takes an advertisement configuration A and outputs a map from $\langle \texttt{prefix, UG} \rangle$ pairs to ingress links. For example, say $R(A) = f_A$, and $f_A(p, \texttt{UG}) = l$ — this notation means that the output of an advertisement configuration A under routing R is a function f_A that tells us that users UG reach prefix p via link l. It could be that a configuration leads to no route for some UG to some prefix. We define a function e such that $e(R(A)(p, \texttt{UG})) = 1$ when there is some route for UG to prefix p under configuration A, and 0 otherwise.

Now suppose the overall metric we want to minimize is G which is a function of both configurations and traffic assignments. Examples include traffic cost, average latency, maximum latency, and their combinations (§3.5). The joint minimization over configurations and traffic assignments can then be expressed as the following.

$$\begin{aligned} \min_{A,w} \quad & G(R(A), w) \\ \text{s.t.} \quad & w(p, \texttt{UG}) \geq 0; \quad A(p,l) \in \{0,1\} \quad \forall \texttt{UG}, p, l \\ & \sum_p w(p, \texttt{UG}) e(R(A)(p, \texttt{UG})) = v(\texttt{UG}) \quad \forall \texttt{UG} \end{aligned} \tag{1}$$

The first constraint requires that traffic assignments be non-negative and that configurations are binary. The second constraint requires that all user traffic $v(\texttt{UG})$ is assigned, and none is assigned to nonexistent paths. Capacity constraints depend on the choice of G.

Solving Equation (1) is challenging because we need to compute $G(R(A), w)$, but measuring $R(A)$ exactly requires advertising prefixes in the Internet, which can only be done infrequently to avoid route-flap-dampening and allow convergence. The optimization problem requires evaluating $G(R(A), w)$ for millions of different A, which could take a hundred years at a rate of advertising one configuration every 20 minutes.

Moreover, even if R were known, Equation (1) is not a tractable optimization problem since (1) G depends on A only through R, which is the Internet's routing rather than an algebraic function of A, and (2) the resulting MIP problem is too large for off-the-shelf solvers.

$$\min_{A} \quad \mathrm{E}_R \left[\min_{w} G(R(A), w) \right] \quad \text{s.t. constraints of Equation (1)} \tag{2}$$

In the remainder of this section we describe each of the three components in Equation (2). The expectation over R represents what we know about routing and how we learn more (§3.4). The outer minimization over A is how we move through configuration space (§3.5.2). The inner minimization over w is how we allocate traffic once routes are fixed (§3.6), and it is where a Service Provider's specific objective, G, plugs in (§5.1.2).

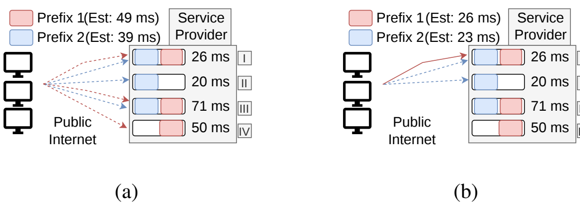


Figure 4: SCULPTOR initially estimates latency from this UG to both the red and blue prefixes with the average over possible ingress latencies (4a). A priori, no prefix is advertised. SCULPTOR then advertises the red prefix and learns that ingress I is preferred over III and IV (4b). SCULPTOR uses this information to refine its latency estimate towards the blue prefix (ingress III is not a preferred option) without advertising the blue prefix, saving time.

3.4 Modeling & Measuring Routes

In Equation (2) we compute expectations over a random process that represents the routing, R, since we cannot exhaustively measure R. Specifically, SCULPTOR models UG paths probabilistically and improves its model over time through relatively few measurements.

3.4.1 Routing Model

A priori, for a given UG towards a given prefix, SCULPTOR treats all reachable ingress link options to which that prefix is advertised as equally likely. One could incorporate prior information about ingress preferences (from economic/routing models, for example), but such complexity is unnecessary for good performance.

We instead learn preferences over time via BGP measurements and observe which ingress the UG takes (§3.4.2). Upon learning that one ingress is preferred over the others receiving that prefix advertisement, we exclude that less-preferred ingress as an option for that UG in all future calculations for all prefixes for which both ingresses are an option. Prior work used a similar routing model [16, 66, 1] but did not extend it to deal with general objectives nor did it treat routing probabilistically.

As we learn which paths UGs prefer the most, our estimate of G on unmeasured configurations concentrates on the true value, allowing us to more efficiently carry out Equation (2). An example of this learning process is shown in Figure 4—SCULPTOR refines a latency estimate towards an unmeasured prefix (blue) using measurements towards other prefixes (red). Even though SCULPTOR has 50% confidence in which path the user takes (*i.e.,* in $R(A)$), SCULPTOR knows that the user's latency towards the blue prefix will be about 23 ms (*i.e.,* either 26 ms or 20 ms) and so SCULPTOR obtains a better estimate of the objective.

3.4.2 Refining the Model

Probabilistic estimates of G could be inaccurate until we learn UG preferences. To refine its model, SCULPTOR periodically measures $R(A)$ on its current configuration (§3.5) by advertising prefixes and measuring UG preferences by observing which routes users take. To use its BGP measurement budget wisely, SCULPTOR makes two design decisions:

What to measure: SCULPTOR measures the *current* configuration (as opposed to random or slightly different ones) since it uses a descent-based optimization (§3.5). This type of optimization compares the current advertisement to many adjacent/different ones, so knowing the current value is much more important than knowing adjacent ones.

When to measure: SCULPTOR measures either when (a) its uncertainty (according to Monte Carlo simulations, §3.6) exceeds a certain threshold or (b) SCULPTOR has not executed a measurement in many learning iterations. There are many possible choices of "uncertainty", we choose entropy [76]. Intelligently choosing when to measure offers a slight performance improvement over a fixed measurement schedule (§5.4.1).

3.5 Searching Over Configurations

We now describe the outer optimization loop in Equation (2) over advertisement configurations, A.

3.5.1 Initializing A

We initialize our configuration to be `anycast` on one prefix and `unicast` on remaining prefixes (*i.e.,* one prefix per site), since both are well-understood configurations with complementary strengths (§2.2). Remaining prefixes in the budget (if they exist) are advertised to random subsets of ingresses.

3.5.2 Gradient Descent

For the reasons discussed in Section 3.3 we cannot find A simply using an off-the-shelf solver. A reasonable choice to solve such problems are descent-based algorithms—ones that compare adjacent options to the current best guess and pick an adjacent option as the next configuration. However, an *additional* challenge in our setting is that we only have noisy estimates of the objective, so placing too much trust in objective estimates of adjacent configurations can lead to poor convergence.

SCULPTOR instead uses *gradient* descent since gradient descent slowly incorporates and remembers information about how useful each advertisement is rather than overcommitting, which is what greedy and random searches used by prior systems do [16, 1].

To apply gradient descent to our problem, we create a real-valued auxiliary variable to A, $\tilde{A}$, with entries between 0 and 1 representing "confidence" values that an advertisement to the corresponding ingress will be beneficial. We threshold $\tilde{A}$ at 0.5 to translate real-valued confidences to whether or not we should advertise a prefix at an ingress (*i.e.,* thresholded $\tilde{A} = A$). For coordinate (p, l), SCULPTOR evaluates the inner objective (§3.6) with $A(p,l)$ set to 1 and then to 0, holding the rest of the (thresholded) configuration fixed. The difference between these two values, scaled by the derivative of a sigmoid at $\tilde{A}$, is the gradient entry (similar to prior work [77]). Since there are too many gradients to compute, we subsample gradient entries and track the largest ones (as in prior work [78]).

We implement SCULPTOR with Nesterov's Accelerated Gradient Descent (§3.5.2) in Python [79]. We use RMSProp with an initial learning rate of 1.0.

Scaling: Gradient descent scales with the product of the number of ingresses and number of prefixes but individual entry computation is independent and therefore parallelizable.

Convergence: Gradient descent converges to a local minimum for bounded objectives [80]. Our evaluations show that SCULPTOR finds good solutions over a wide range of emulated topologies (§4.2) and converges quickly with thousands of UGs and ⟨peering, prefix⟩ pairs (§5). Allocating higher prefix budgets and adding richer advertisement capabilities (*e.g.,* BGP community tagging) can lead to convergence to a better minimum, which is an area for future work.

3.5.3 Termination

SCULPTOR stops gradient descent either when the BGP measurement budget (*i.e.,* time budget) has been exhausted or the average change in objective over iterations falls below a small threshold. We stop the optimization when measurement budget has been exhausted since SCULPTOR may make poor decisions if it has a bad estimate of the routing function.

3.6 Allocating Traffic

This section specifies the inner minimization over traffic allocations in Equation (2).

Given our routing model (§3.4.1) and current advertisement (§3.5), we Monte Carlo sample routing functions R. Each sample independently draws, for every UG and prefix, one ingress uniformly from the ingresses the model still considers possible for that pair (§3.4.1). For each sample we then solve the traffic allocation exactly with a general-purpose solver, and the expectation in Equation (2) is the average of those solutions. In practice we find that a few Monte Carlo samples per gradient entry is sufficient.

Given a sampled routing realization, solving for the optimal traffic allocation is then a problem in classical (*i.e.,* intradomain) traffic engineering. We solve traffic allocations (§3.6) with SciPy HIGHS [81].

Scaling: This component scales with the number of Monte Carlo samples and the scaling behavior of each traffic allocation, which depends on the objective, G.

Convergence: depends on the objective, G. SCULPTOR accepts any G for which, given fixed routes, the allocation problem is efficiently solvable. All objectives that we evaluate in Section 5.1.2 are linear programs.

4 Implementation

We prototype SCULPTOR on the PEERING testbed [21], which is available at 32 Vultr cloud sites. We describe how we built SCULPTOR on the real Internet and how we *emulate* a Service Provider including their clients, traffic volumes, and resource capacities.

In practice, Service Providers can use SCULPTOR *safely* by only shifting production traffic onto prefixes after SCULPTOR's announcement exploration has converged and by integrating SCULPTOR into automated route configuration frameworks. More details about how a Service Provider may do so are in Appendix B.

4.1 Evaluation Limitations

We are not a Service Provider and so could not obtain actual volumes/capacities—latency and routes are the real Internet phenomenon we, as researchers, are able to measure. We *do* evaluate SCULPTOR on hundreds of routing and capacity provisioning scenarios to demonstrate that its benefits are not particular to any deployment. Prior interdomain routing work that evaluates capacity/congestion dynamics *was conducted by Service Providers* [82, 83, 5, 59, 84, 85, 35, 6], whereas prior work from academics used similar methodology to our own [86, 53, 87].

4.2 Emulating Client Traffic

We measure actual client latency from IP addresses to our PEERING prototype as in prior work [1], selecting targets according to assumptions about Vultr cloud's client base.

We first tabulate a list of 5M IPv4 targets that respond to ping via probing each /24. Vultr informs cloud customers of which prefixes are reachable via which peers, and we use this information to tabulate a list of peers and clients reachable through those peers. We then measure latency from all clients to each peer individually by advertising a prefix solely to that peer using Vultr's BGP action communities and pinging clients from Vultr. We also measure performance from all clients to all providers individually, as providers provide global reachability.

In our evaluations, we limit our focus to clients who had a route through at least one of Vultr's direct peers (we exclude route server peers [88]). Vultr likely peers with networks with which it exchanges a significant amount of traffic [35], so clients with routes through those peers are more likely to be "important" to Vultr. We found 700k /24s with routes through 1086 of Vultr's direct ingresses. We removed clients whose lowest latency to Vultr was 1 ms or less, as these were assumed to be addresses related to infrastructure, leaving us with measurements from 666k /24s to 825 Vultr ingresses.

We emulate traffic volumes in two ways: First, we randomly choose the total traffic volume of a site as a number between 1 and 10 and then divide volume up randomly among clients that anycast routes to that site. Client volumes in a site are set to be within one order of magnitude of each other. While potentially unrealistic, testing many different random volumes demonstrates SCULPTOR's robustness to assumptions about a particular distribution. We avoid Zipf-like distributions as they lead to trivial solutions where a small fraction of sites dominate importance.

Second, we derive realistic volumes using APNIC AS populations [89], shown to be representative of Service Provider traffic volumes [90]. We assign each AS its aggregate population and divide it evenly among its clients.

We found that the high-level results shown in Section 5 were the same using both emulated and APNIC volumes, and so report results for emulated traffic.

4.3 Deployments

Experiments in the Internet. We assess how SCULPTOR performs on the Internet using RIPE Atlas probes [91], which represent a subset of all clients. RIPE Atlas allows us to measure paths (and thus ingress links) to prefixes we announce from PEERING, which SCULPTOR needs to refine its model (§3). We limit the scale of our deployment to 10 sites to avoid reaching RIPE Atlas daily probing limits (15k traceroutes/day). We prioritize geographic density over coverage to create a challenging routing surface with subtle unicast differences. These 10 sites include major hubs across North America, Europe, and South America. Choosing RIPE Atlas probes to maximize network coverage and geographic diversity, we select probes from 972 networks in 38 countries which have paths to 484 unique ingresses. Each probe has paths via approximately 60 ingresses. We use 12 prefixes and change advertisements at most once per hour to avoid route-flap-dampening.

Emulations. We also evaluate SCULPTOR by emulating user paths which allows us to conduct more extensive evaluations, as experiments take less time and use clients in more networks. We compute solutions over many random routing preferences, demands, and subsets of sites to demonstrate that SCULPTOR's benefits are not limited to a specific deployment. We evaluate SCULPTOR over deployments of size 3, 5, 10, 15, 20, 25, and 32 sites. Each size deployment is run at least 10 times with random subsets of UGs, UG demands, and routing preferences. The distribution of the number of /24s per peer is Pareto-like, so we consider random subsets of /24s through each ingress in a way that balances the number of unique /24s per ingress. Over all scenarios, we consider paths from clients in 52k prefixes (representing 31% of APNIC population [89]) to 873 ingresses. We use one tenth of the number of ingresses as the number of prefixes in our budget (10 prefixes for 3 site deployments, 60 prefixes for 32 site deployments). Prior work found that using tens of prefixes to improve performance was a reasonable cost [16, 1].

4.4 Setting Resource Capacities

We assume that resource capacities are overprovisioned proportional to their usual load. However, we do not know the usual load of Vultr links and cannot even determine which peering link that traffic to one of our prefixes arrives on, as Vultr does not give us this information. (This limitation exists since we are not a Service Provider, but a Service Provider could measure this using IPFIX, for example.) We overcome this limitation using two methods corresponding to our two deployments in Section 4.3.

Experiments in the Internet. For our first method of inferring client ingress links, we advertise prefixes into the Internet using the PEERING testbed [21], and measure actual ingress links to those prefixes using traceroutes from RIPE Atlas probes [91]. Specifically, we perform IP to AS mappings and identify the previous AS in the path to Vultr. This approach has limited evaluation coverage, as RIPE Atlas probes are in a few thousand networks. In cases where we cannot infer the ingress link from a traceroute, we use the closest latency from the traceroute to the clients' (known) possible ingresses. For example, if an uninformative traceroute's latency was 40 ms to Vultr's Atlanta site and a client was known to have a 40 ms path through AS1299 at that site, we say the ingress link was AS1299 at Atlanta.

Emulations. The second method we use to infer ingress links is emulating user paths by assuming we know all user routing preferences (§3.2). We use a preference model where clients prefer peers over providers, and clients have a preferred provider. When choosing among multiple ingresses for the same peer/provider, clients prefer the lowest-latency option. We also add in random violations of the model. This approach allows us to evaluate our model on *all* client networks but may not represent actual routing conditions, though prior work found it held in 90% of the cases they studied. We conducted a sensitivity analysis on this model/assumption and found that SCULPTOR's performance gains are robust to variations (*e.g.*, random preferences).

Given either method of inferring client ingress links (RIPE Atlas/emulations), we then measure paths to an anycast prefix and assign resource capacities as some overprovisioned percentage of this catchment. (Discussions with operators from Service Providers suggested that they overprovision using this principle.) We report results for an overprovisioning rate of 10%, but find similar takeaways for 10% through 50%.

5 Evaluation

5.1 Evaluation Setting

5.1.1 Baseline Solutions

anycast.: A single prefix announcement to all peers/providers at all sites, which is a common configuration used by Service Providers today [92, 93, 46, 94, 95, 96, 49].

unicast.: A single prefix announcement to all peers/providers at each site (one per site). Another common configuration used in today's deployments [50, 51, 52].

AnyOpt & PAINTER.: Two proposed strategies for reducing steady-state latency compared to anycast [16, 1]. AnyOpt was one of the first works to systematically optimize ingress traffic, while PAINTER built on that effort. (We do not compute the solution for AnyOpt for our evaluations on the Internet since AnyOpt did not perform well compared to any other solution in our emulations, and since AnyOpt takes a long time to compute.)

One-per-Peering.: A unique prefix advertisement to each peer/provider, so many possible paths are always available from users. This solution serves as our performance upper-bound, even though it is prohibitively expensive. (We do not know an optimal solution with fewer prefixes.)

We compute both average overall latency and the fraction of traffic within 10 ms (very little routing inefficiency), 50 ms (some routing inefficiency), and 100 ms (lots of routing inefficiency) of the One-per-Peering solution for each advertisement configuration, as these statistics provide a more informative measure of latency improvement than averages.

5.1.2 Objectives Evaluated

While our approach likely accommodates other objectives [59, 35, 97, 1, 16, 52, 20, 17, 18, 98], it provides practical utility at the expense of a formal characterization of feasible functions, similar to prior gradient descent work. We write down the objective for (1) as an example (our other objectives often have similar structure).

Latency + Maximum Link Utilization. Under our model (§3.2.1), steady-state path latency for a UG to a prefix is uniquely determined by UG and the peering link over which the UG ingresses (but latency may change over time, which we evaluate in Section 5.3.2). Let the latency for a user UG via a link l be $\mathcal{L}(\mathtt{UG}, l)$. G is then given by the following.

$$G(R(A),w) = \frac{1}{\sum_{\mathtt{UG}} v(\mathtt{UG})} \sum_{p,\mathtt{UG}} \mathcal{L}(\mathtt{UG}, R(A)(p,\mathtt{UG}))w(p,\mathtt{UG}) + \beta M$$
$$\text{s.t.} \quad \frac{\sum_{R(A)(p,\mathtt{UG})=l} w(p,\mathtt{UG})}{c(l)} \leq M \quad \forall l \tag{3}$$

For link capacity $c(l)$ minimizing Equation (3) forces M to be the maximum link utilization (MLU). β weights a tradeoff between using uncongested links/sites and low propagation delay and is set by the Service Provider based on their goals. We first solve with $M = 1$ to see if we can allocate traffic to paths with zero overloading, which may not be possible for all A.

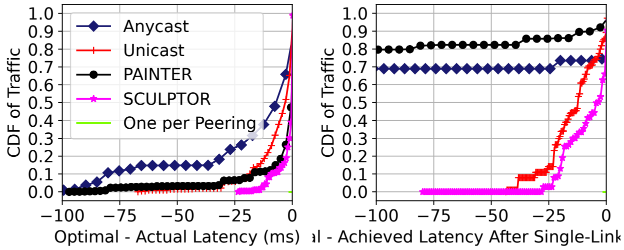
Dynamic Traffic Failover. To encourage robustness under unseen conditions (*e.g.,* failures), we add $\sum_l \alpha_l G(R(A * F_l), w)$ to G. Binary vectors F_l are defined so that multiplying advertisements, A, by these vectors models withdrawal/failure on a link/site (*i.e.,* zeroing out the corresponding components). This regularizer encourages solutions with good primary and backup paths. We choose $alpha_l = 0.1$ in our evaluations. This objective assumes that the Service Provider can rebalance traffic quickly using a technology like MPQUIC.

Static Traffic Failover The same as **Regularizing For Failure Robustness.**, but assuming that prefix assignments are fixed during failure. This objective hence assumes that the Service Provider uses a slow traffic rebalancing technology like DNS.

Latency Sensitive Services. Some services have very tight latency requirements (*e.g.,* $\leq$ 10 ms) [3]. This objective aims to maximize the amount of traffic within 10ms of its One-per-Peering latency. (We choose the difference with One-per-Peering latency as opposed to absolute latency to isolate SCULPTOR's goal of minimizing routing inefficiency.)

Traffic Classes. Similar to private WAN traffic engineering [18, 17], we consider routing traffic with distinct performance requirements. We consider a scenario where one traffic class (high-priority) should be routed with low latency, and the other (low-priority) should not congest the high-priority traffic. An additional challenge in the interdomain setting is that we cannot use priority queueing to ensure that high-priority traffic is not congested since we do not control queue behavior on the path. The objective is a weighted combination of the average latency of high-priority traffic and the amount of high-priority traffic that is congested. We also set a limit on the maximum overload on any link so that not all low-priority traffic lands on one link, as this solution would lead to poor low-priority goodput.

Traffic Cost Across Sites. We also consider the scenario where each site has a cost proportional to the traffic volume arriving at the site. The cost per unit can vary across sites. This objective is a simple model of scenarios where Service Providers want to minimize power/carbon/peering infrastructure cost (which can vary by region) or minimize the use of unreliable sites (*e.g.,* dirty power). We minimize the weighted sum of average latency and average site cost. Our goal is to demonstrate that our approach supports trading off across these types of goals, not to argue that our simple cost model is correct.



(a) (b)

Figure 5: SCULPTOR outperforms other solutions on the Internet deployment, reducing steady-state latency (a) and congestion during link failures (b).

5.2 On-Internet Experiments

We evaluate (a) `Latency + Maximum Link Utilization` and (b) `Dynamic Traffic Failover` on our Internet deployment (§4.3). For context, at the scale of Service Providers that serve trillions of requests per day, improving a few percent of traffic by tens of milliseconds represents a significant improvement. Service Providers recently emphasized that small percentage gains are important [99, 100].

5.2.1 Steady-State Latency and MLU

Figure 5a shows a CDF of the difference in latency between each solution and One-per-Peering for all UG*s*, weighted by traffic. SCULPTOR outperforms other solutions and is only 2.0 ms worse than (the unreasonably expensive) One-per-Peering solution on average, whereas the next best solution, PAINTER, is 5.5 ms worse. SCULPTOR also serves 91.8% of traffic within 10 ms of the latency with which One-per-Peering serves it, whereas the same is true for only 88% of traffic for PAINTER.

5.2.2 Failure Robustness

Figure 5b demonstrates that SCULPTOR offers lower latency paths for more UG*s* during single link failure in realistic routing conditions. On average, SCULPTOR is 7.9 ms higher latency than One-per-Peering, compared to unicast which is 14.2 ms higher than One-per-Peering. PAINTER struggles to find sufficient capacity for UG*s*, overloading 69.6% of traffic.

Site failures show similar results. On average, SCULPTOR is 13.1 ms higher latency than One-per-Peering, while the next-best solution, unicast, is 25.1 ms higher. PAINTER again performs poorly, with 95% of traffic overloaded during site failure.

5.3 Emulated Experiments

We extensively evaluate SCULPTOR against other solutions across all objectives in our emulated deployments. Table 1 demonstrates that SCULPTOR outperforms all other solutions on deployments. Each objective was trained and computed on 10 emulated deployments.

5.3.1 Latency + Maximum Link Utilization

Average latency for SCULPTOR is on average 1.3 ms worse than One-per-Peering while the next-best solution (PAINTER) is on average 3.4 ms worse than SCULPTOR. Interestingly, unicast (5.1 ms worse than One-per-Peering) performs better than AnyOpt (17.1 ms worse than One-per-Peering) which may be because AnyOpt was designed to minimize latency without capacity constraints while unicast naturally provides many high-capacity paths to users. SCULPTOR obtains the same MLU as One-per-Peering in all cases.

5.3.2 Dynamic Traffic Failover

We next assess SCULPTOR's ability to robustly handle failures where the Service Provider has a technology for rebalancing traffic across prefixes quickly after the failure (*e.g.,* using MPQUIC). Failures here could represent excessive DDoS traffic on the link/site (thus using the link/site as a sink for the attack), physical failure, resource draining, changes in latency on a path, congestion in the public Internet, and planned maintenance. Table 1 demonstrates that SCULPTOR shifts traffic without overloading alternate links/sites more effectively than other solutions, *without reactive BGP changes* which could cause further failure (§2.3).

In solving for traffic allocations during failure scenarios, links may be overloaded. We say all traffic arriving on a congested link is congested. We separately note the fraction of traffic that lands on congested links and do not include it in average latency computations, but say such traffic does *not* satisfy 10 ms, 50 ms, or 100 ms objectives.

Performance Under Ingress/Site Failure We fail single links and find that and find that most solutions have zero congestion since, with 32 sites, there are many backup options. SCULPTOR, however, is only 6 ms worse than One-per-Peering on average during these failures while unicast (the next-best solution) is 17 ms worse on average. During more extreme site failures, SCULPTOR only congests 2.22% of traffic on average whereas PAINTER congests 14% of traffic.

Efficient Infrastructure Utilization In training for failure robustness, we find that SCULPTOR enables distributing higher-than-normal load over existing infrastructure. We quantify this improved infrastructure utilization under two realistic traffic patterns — flash crowds and diurnal patterns.

Methodology. We define a flash crowd as traffic increase for users in a region. Examples include content releases that spur downloads in a particular region and localized DDoS

Objectives	Latency + Maximum Link Utilization (§§ 5.2.1 and 5.3.1)		Dynamic Traffic Failover (§§ 5.2.2 and 5.3.2)			Static Traffic Failover (§5.3.3)		Latency Sensitive Services (§5.3.4)	Traffic Classes (§5.3.5)		Traffic Cost Across Sites (§5.3.6)
Method	Latency (ms)	Maximum link utilization (vs anycast)	Ingress fail: latency (ms) / % cong	Site fail: latency (ms) / % cong	Flash Crowd/Diurnal Intensity (vs anycast)	Ingress fail: latency (ms) / % cong / % no-route	Site fail: latency (ms) / % cong / % no-route	% beyond 10ms of optimal	HPrio latency (ms)	Crit bulk ratio	Wgt avg site cost (vs anycast)
`One-per-Peering` [1]	29.1	0.966	29.4 / 0.00	30.9 / 5.65	1.39 / 1.44	29.2 / 0.00 / 0.00	32.1 / 0.50 / 0.00	12.9	29.0	5.62	0.918
`SCULPTOR`	**30.4**	**0.967**	**30.4 / 0.00**	**32.1 / 5.18**	**1.26 / 1.33**	**34.0 / 0.04 / 0.00**	**36.2 / 3.94 / 0.00**	**16.3**	**31.3**	**4.35**	**0.941**
`PAINTER`	33.7	0.995	33.6 / 0.00	37.0 / 8.29	1.06 / 1.08	40.3 / 0.19 / 0.00	40.6 / 7.32 / 0.00	21.0	33.7	1.29	0.980
`unicast`	34.2	0.978	34.7 / 0.00	36.8 / 5.44	1.26 / 1.29	51.9 / 0.38 / 0.00	49.7 / 6.58 / 0.00	24.0	34.2	2.74	0.953
`AnyOpt`	46.2	0.999	47.4 / 0.20	37.9 / 21.77	1.00 / 1.04	48.4 / 0.16 / 0.00	46.9 / 6.55 / 0.00	31.8	46.1	1.19	0.992
`anycast`	51.2	1.000	51.8 / 0.37	50.1 / 6.38	1.00 / 1.00	51.9 / 0.38 / 0.00	49.7 / 6.58 / 0.00	32.7	51.7	0.25	1.000

[1] An unrealistic optimal, included for comparison.

Table 1: Performance of all methods across objectives. `SCULPTOR` outperforms all other methods on every metric.

attacks. Since increased demand is localized, we can spread excess demand to other sites, which is a cheaper option than installing more capacity (see Section 3.2.1 for our cost model). Links are still provisioned 10% higher than `anycast` traffic volume as in the rest of Section 5.3.2.

We identify each client with a single "region" corresponding to the site at which they have the lowest possible latency ingress link. For each region individually, we then scale each client's traffic in that region by M% and compute traffic to path allocations. If there are S sites in the deployment, we thus compute S separate allocations per M value where each allocation assumes inflated traffic in exactly one region, but all the allocations are across a single set of routes calculated based on the original (non-flash) traffic. For a target region, we increase M until any link experiences overloading, then find the lowest such M value across regions. For example, if a 60% increase in traffic for Atlanta clients creates overload (and no values < 60% did for any region), we call M = 160% the critical value of M.

We use a similar methodology for diurnal traffic analysis. We define a diurnal effect as a traffic pattern that changes volume according to the time of day. Diurnal effects might be different for different Service Providers, but a prior study from a Service Provider demonstrates that these effects can cause large differences between peak and mean site volume [20]. We sample diurnal patterns from that publication and apply them to our own traffic. We group sites in the same time zone and assign traffic in different time zones different multipliers — in "peak" time zones we assign a multiplier of M and in "trough" time zones a multiplier of $0.1M$. The full curve is shown in Appendix C. Similar to our flash crowd analysis, we increase M until at least one link experiences overloading at least one hour of the day.

Results. Table 1 shows the average critical M value over deployments that cause overloading under flash crowds and diurnal effects computed using emulated deployments. `SCULPTOR` is able to route traffic with **TBD: 33.64**× normal volume in all regions, whereas the same is true for 30.97% for `PAINTER`. Similarly, `SCULPTOR` achieves a diurnal intensity of 13.07 for diurnal traffic whereas the next-bast solution is `unicast` with 12.66.

5.3.3 Static Traffic Failover

We perform a similar evaluation as `Dynamic Traffic Failover` except that, during failure, users are frozen to their prefixes (possibly since the Service Provider uses DNS, which is slow to react). We see similar findings with latency staying **TBD: 5**ms of `One-per-Peering` and **TBD: 0.3**% congestion whereas `PAINTER`, the next-best solution, is **TBD: 10**ms worse than `One-per-Peering` and has **TBD: 10**% congestion.

5.3.4 Latency Sensitive Services

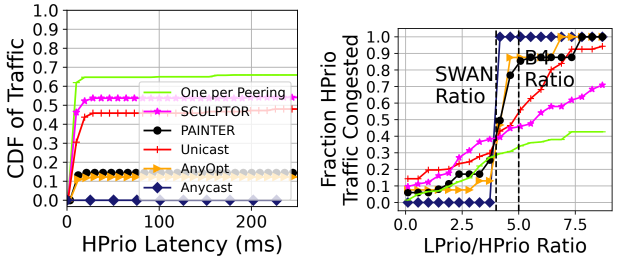
We also evaluate `SCULPTOR`'s ability to enable latency-sensitive services that have hard latency requirements such as VR/AR with a requirement of 10 ms [3]. Specifically, we try to maximize the amount of traffic to within 10 ms of its lowest latency option. This objective is highly non-linear (a step function) and so challenging to optimize.

Table 1 demonstrates that `SCULPTOR` pushes 4% more traffic globally to within this threshold than `PAINTER` and, compared to `anycast`, pushes us 83% of the way to the `One-per-Peering` solution.

5.3.5 Traffic Classes

We also evaluate `SCULPTOR`'s ability to satisfy multiple traffic classes. We split traffic into high and low-priority. The objective is to route high-priority traffic to low-latency routes while limiting congestion on those routes from low-priority traffic. We do not penalize cases where low-priority is congested, but do limit the maximum amount of any traffic on a link to 10× the capacity of the link to avoid solutions where all low-priority is placed on one link, as this solution would lead to low goodput for low-priority traffic (in practice, `UG`s would lower sending rates in response to congestion). We solve `SCULPTOR` on a single emulated 32 site deployment.

In Figure 6b we vary the amount of low-priority traffic as a multiple of high-priority traffic volume and compute the fraction of high-priority traffic congested. Links are provisioned to 5× the high-priority traffic volume (different from Section 5.3.2), as that is roughly the `LPrio`/`HPrio` ratio reported in prior work [18, 17]. There is insufficient global capacity to route all traffic for all `LPrio`/`HPrio` over 4.0, thus



(a) (b)

Figure 6: SCULPTOR routes high-priority traffic with low latency (6a) and minimal congestion from low-priority traffic (6b). We inferred LPrio/HPrio ratios for SWAN [18] and B4 [17] from those papers.

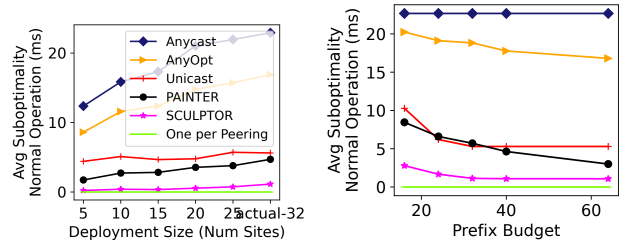
the jump in anycast congestion in Figure 6b. SCULPTOR finds strategies that allow us to route more low-priority traffic with less congestion than all other approaches. For example, when LPrio/HPrio = 5, SCULPTOR achieves half as much congestion as PAINTER and unicast.

Figure 6a also shows that SCULPTOR routes traffic with lower latency than other solutions (intercepts at the right of the graph show fractions of high-priority traffic not congested). Figure 6a uses LPrio/HPrio = 4, a midpoint between the ratios seen in SWAN [18] and B4 [17].

5.3.6 Traffic Cost Across Sites

We also evaluate SCULPTOR's ability to jointly minimize both latency and per-site traffic cost, since different sites may cost more or less to serve traffic due to, for example, the availability of power. We solve SCULPTOR on three emulated 32 site deployments(§4.3), using different per-site costs: (a) completely random per-site costs, (b) a model of expensive vs cheap sites where half the sites were 1.2× the cost of other sites, and (c) a model of per-site cost based on the grams of CO_2 emitted per kWh of electricity consumed at the site, similar to prior work [101, 102]. We do not claim to have realistic site cost values but use different site-cost choices to demonstrate that SCULPTOR's benefits are not specific to a configuration.

In our emulations, SCULPTOR consistently found solutions closer to the One-per-Peering one than all other approaches. Under cost model (b), its objective gap to One-per-Peering is 1.5× smaller than the next-best solution, PAINTER; under cost model (c), the gap is likewise 3× smaller. When we increase the weight on site-cost savings by 100×, we observe that SCULPTOR selects solutions with 24ms higher average latency in exchange for using sites with 12% lower cost. Compared to the optimal One-per-Peering, the resulting objective gap for SCULPTOR is 8× smaller than that of PAINTER. This demonstrates that SCULPTOR keeps cost low even if Service Provider goals change.



(a) (b)

Figure 7: Micro evaluations.

5.4 Micro-Evaluations

5.4.1 Ablation Study

SCULPTOR consists of several methodological changes over prior work (AnyOpt, PAINTER) that enable it to achieve its goals. All of them aim to solve Equation (1), but model that objective function and search through the space in different ways. To break down how much each change matters, we conduct an ablation study where we randomly emulated 10 10-site deployments and optimized Dynamic Traffic Failover (§5.1.2) with different solvers.

Methodology. First, we evaluate PAINTER which uses a routing preference model and estimates objectives as the average objective value over ingresses with greedy search. Then we evaluate "Monte Carlo" which replaces objective estimation with our methodology in Section 3.4.1. Then we evaluate with "Monte Carlo + descent" which is the descent outlined in **??** without the gradient/memory. Then we evaluate "Monte Carlo + gradient descent" and, finally, we add in smart measurements which is the final component of SCULPTOR (§3.4.2).

Results. Normalizing PAINTER to 0% and One-per-Peering to 100%, we find that (on average), Monte Carlo gives 40% of the benefit, Monte Carlo + descent gives 78% of the benefit, and both Monte Carlo + gradient descent and full SCULPTOR (with smart measurements) give 91% of the benefit. Smart measurements, however, allow us to achieve the same benefit with **TBD: 10**% fewer measurements (*i.e.,* converging quicker), on average.

5.4.2 Effect of Deployment Size & Prefix Budget

We vary deployment size and prefix budget separately for at least 10 deployments at each budget/size while optimizing Dynamic Traffic Failover. We are primarily interested if the difference between SCULPTOR and the baselines grows, shrinks, or holds as these parameters vary.

Deployment Size For each size, we select a random subset of all sites and emulate a deployment of that size using all ingresses at that site. We find that all metrics generally *hold*, and grow significantly compared to PAINTER. For example, SCULPTOR's steady-state latency grows 1 ms compared to One-per-Peering but grows 5 ms for PAINTER compared to One-per-Peering, nearing unicast performance.

Prefix Budget We evaluate the effect of prefix budget on our failure robustness metric. For each deployment size, we select that many sites and train+evaluate SCULPTOR against the baselines. We are primarily interested if the difference between SCULPTOR and the baselines grows, shrinks, or holds.

6 Related Work

Egress Traffic Engineering. Prior work noted that traffic from Service Providers to users sometimes experienced sub-optimal performance due to BGP's limitations [83, 35, 85]. SCULPTOR works in tandem with these systems, similarly working with BGP, but to engineer ingress traffic. Other work also shifts traffic to other links/sites to lower cost [84, 59, 20]. SCULPTOR adapts this idea in a new way but instead uses the public Internet to carry traffic to lower costs.

Ingress Traffic Engineering. We evaluate (§5) and discuss (§2.3) differences to other ingress traffic engineering [46, 82, 16, 48, 49, 6, 1, 103]. FastRoute coarsely balances users across anycast rings to respond to changing conditions [5], giving Service Providers much less control over the specific path users take. PECAN [104] and Tango [105] exhaustively expose paths between endpoints and so may provide resilience to changing conditions if the right paths were exposed. However, exposing all the paths does not scale to our setting since there are too many paths to measure. Recent work balances ingress traffic across sites during failure [87], or optimizes latency by predicting traffic how traffic routes [106, 103], but it is unclear how to extend these methodologies to other objectives.

Other prior work built a BGP playbook to mitigate DDoS attacks [86], but it is unclear if those strategies scale to larger Service Providers (they tested on a few sites/providers). SCULPTOR works for Service Providers with thousands of peering links. Other work/companies create overlay networks and balance load through paths in these overlay networks to satisfy latency requirements [107, 108, 109, 110, 38]. SCULPTOR works alongside these overlay networks by advertising the external reachability of these nodes to create better paths through the overlay structure.

Intradomain Failure Planning. Service Providers have shown interest in reducing the frequency/impact of failures in their global networks [111, 32, 33, 15]. SCULPTOR works alongside these systems by, for example, enabling Service Providers to shift traffic away from failed components/regions while still retaining good performance. Prior work also shifted traffic during peak times to limit cost/congestion [20], but did so using a private backbone. SCULPTOR's benefits are orthogonal to this prior work and useful for Service Providers without private backbones, as they use the public Internet to realize the same result. Other prior work tried to plan intradomain routes to minimize the impact of k-component failures [97, 98, 112, 113, 20]. SCULPTOR solves different challenges that arise due to the lack of visibility/control into potential paths and their capacities in the interdomain setting.

7 Discussion & Conclusions

Unilateral control over intradomain traffic has enabled technology that improves performance, reliability, and flexibility, including fast reroute for quick recovery from failure [114], virtual routing and forwarding for flexible traffic engineering [115], and virtual output queueing for differentiated service [23]. Conversely, interdomain proposals for differentiated services (*e.g.,* intserv/diffserv [116], L4S [117]) and multipath routing (MIRO [118]) have seen little deployment.

SCULPTOR requires no Internet-wide changes. Instead, it leverages black-box modeling and BGP flexibility to unlock a provider's unused global capacity. SCULPTOR is a step towards providing Service Providers with programmable interdomain networking primitives enabling the performance that modern services demand.

References

[1] Thomas Koch, Shuyue Yu, Ethan Katz-Bassett, Ryan Beckett, and Sharad Agarwal. [n.d.]. PAINTER: Ingress Traffic Engineering and Routing for Enterprise Cloud Networks. In *SIGCOMM 2023*.

[2] Bhaskar Kataria, Palak LNU, Rahul Bothra, Rohan Gandhi, Debopam Bhattacherjee, Venkata N Padmanabhan, Irena Atov, Sriraam Ramakrishnan, Somesh Chaturmohta, Chakri Kotipalli, et al. 2024. Saving Private WAN: Using Internet Paths to Offload WAN Traffic in Conferencing Services. *arXiv preprint arXiv:2407.02037* (2024).

[3] Nitinder Mohan, Lorenzo Corneo, Aleksandr Zavodovski, Suzan Bayhan, Walter Wong, and Jussi Kangasharju. [n.d.]. Pruning Edge Research With Latency Shears. In *HotNets 2020*.

[4] Jan-Philipp Stauffert, Florian Niebling, and Marc Erich Latoschik. 2018. Effects of latency jitter on simulator sickness in a search task. In *2018 IEEE conference on virtual reality and 3D user interfaces (VR)*.

[5] Ashley Flavel, Pradeepkumar Mani, David Maltz, Nick Holt, Jie Liu, Yingying Chen, and Oleg Surmachev. [n.d.]. FastRoute: A Scalable Load-Aware Anycast Routing Architecture for Modern CDNs. In *NSDI 2015*.

[6] Michael Markovitch, Sharad Agarwal, Rodrigo Fonseca, Ryan Beckett, Chuanji Zhang, Irena Atov, and Somesh Chaturmohta. [n.d.]. TIPSY: Predicting Where Traffic Will Ingress a WAN. In *SIGCOMM 2022*.

[7] Giovane C. M. Moura, Ricardo de Oliveira Schmidt, John Heidemann, Wouter B. de Vries, Moritz Muller, Lan Wei, and Cristian Hesselman. [n.d.]. Anycast vs. DDoS: Evaluating the November 2015 Root DNS Event. In *IMC 2016*.

[8] Matthias Wichtlhuber, Eric Strehle, Daniel Kopp, Lars Prepens, Stefan Stegmueller, Alina Rubina, Christoph Dietzel, and Oliver Hohlfeld. 2022. IXP scrubber: learning from blackholing traffic for ML-driven DDoS detection at scale. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 707–722.

[9] Duncan Riley. 2023. Internet's rapid growth faces challenges amid rising denial-of-service attacks. https://siliconangle.com/2023/09/26/internets-rapid-growth-faces-challenges-amid-rising-ddos-attacks

[10] Giovane CM Moura, John Heidemann, Moritz Müller, Ricardo de O. Schmidt, and Marco Davids. [n.d.]. When the dike breaks: Dissecting DNS defenses during DDoS. In *IMC 2018*.

[11] Matthew Prince. 2013. The DDoS That Knocked Spamhaus Offline (And How We Mitigated It). https://blog.cloudflare.com/the-ddos-that-knocked-spamhaus-offline-and-ho

[12] Sam Kottler. 2018. February 28th DDoS Incident Report. https://github.blog/2018-03-01-ddos-incident-report/

[13] Chris Stokel-Walker. 2021. Case study: How Akamai weathered a surge in capacity growth. https://increment.com/reliability/akamai-capacity-growth-surge/

[14] Satyajeet Singh Ahuja, Varun Gupta, Vinayak Dangui, Soshant Bali, Abishek Gopalan, Hao Zhong, Petr Lapukhov, Yiting Xia, and Ying Zhang. [n.d.]. Capacity-efficient and uncertainty-resilient backbone network planning with hose. In *SIGCOMM 2021*.

[15] Bingzhe Liu, Colin Scott, Mukarram Tariq, Andrew Ferguson, Phillipa Gill, Richard Alimi, Omid Alipourfard, Deepak Arulkannan, Virginia Jean Beauregard, Patrick Conner, et al. [n.d.]. CAPA: An Architecture For Operating Cluster Networks With High Availability. In *NSDI 2024*.

[16] Xiao Zhang, Tanmoy Sen, Zheyuan Zhang, Tim April, Balakrishnan Chandrasekaran, David Choffnes, Bruce M. Maggs, Haiying Shen, Ramesh K. Sitaraman, and Xiaowei Yang. [n.d.]. AnyOpt: Predicting and Optimizing IP Anycast Performance. In *SIGCOMM 2021*.

[17] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, et al. [n.d.]. B4: Experience with a Globally-Deployed Software Defined WAN. In *SIGCOMM 2013*.

[18] Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill, Mohan Nanduri, and Roger Wattenhofer. [n.d.]. Achieving High Utilization with Software-Driven WAN. In *SIGCOMM 2013*.

[19] Congcong Miao, Zhizhen Zhong, Yunming Xiao, Feng Yang, Senkuo Zhang, Yinan Jiang, Zizhuo Bai, Chaodong Lu, Jingyi Geng, Zekun He, et al. 2024. MegaTE: Extending WAN Traffic Engineering to Millions of Endpoints in Virtualized Cloud. (2024).

[20] David Chou, Tianyin Xu, Kaushik Veeraraghavan, Andrew Newell, Sonia Margulis, Lin Xiao, Pol Mauri Ruiz, Justin Meza, Kiryong Ha, Shruti Padmanabha, et al. [n.d.]. Taiji: Managing Global User Traffic for Large-Scale Internet Services at the Edge. In *SOSP 2019*.

[21] Brandon Schlinker, Todd Arnold, Italo Cunha, and Ethan Katz-Bassett. [n.d.]. PEERING: Virtualizing BGP at the Edge for Research. In *CoNEXT 2019*.

[22] VULTR. 2023. VULTR Cloud. https://vultr.com/

[23] Juniper Networks. 2023. Understanding CoS Virtual Output Queues (VOQs). https://juniper.net/documentation/us/en/software/junos/traffic-mgmt-qfx/topics/concept/cos-qfx-series-voq-understanding.html

[24] Mordor Intelligence. 2022. Network as a Service Market - Growth, Trends, COVID-19 Impact, and Forecasts. https://mordorintelligence.com/industry-reports/network-as-a-service-market-growth-trends-and-forecasts

[25] Krishan Arora. 2023. The Gaming Industry: A Behemoth With Unprecedented Global Reach. https://forbes.com/sites/forbesagencycouncil/2023/11/17/the-gaming-industry-a-behemoth-with-unprecedented-global-reach

[26] Cloudflare. 2024. Cloudflare Workers. https://workers.cloudflare.com/

[27] Fastly. 2024. Fastly Compute. https://fastly.com/products/edge-compute

[28] Edgecast. 2020. The CDN Edge brings Compute closer to where it is needed most. https://edgecast.medium.com/the-cdn-edge-brings-compute-closer-to-where-it-is-needed-most-d4a3f4107b11

[29] Google Cloud. 2024. Cloud Video Intelligence API. https://cloud.google.com/video-intelligence

[30] Microsoft. 2024. Emirates Global Aluminiium cuts cost of manufacturing AI by 86 percent with the introduction of Azure Stack HCL. https://customers.microsoft.com/en-us/story/1777264680029793974-ega-azure-arc-discrete-manufacturing-en-united-arab-emirates

[31] Yuchen Jin, Sundararajan Renganathan, Ganesh Ananthanarayanan, Junchen Jiang, Venkata N Padmanabhan, Manuel Schroder, Matt Calder, and Arvind Krishnamurthy. [n.d.]. Zooming in on wide-area latencies to a global cloud provider. In *SIGCOMM 2019*.

[32] Ramesh Govindan, Ina Minei, Mahesh Kallahalla, Bikash Koley, and Amin Vahdat. [n.d.]. Evolve or die: High-availability design principles drawn from googles network infrastructure. In *SIGCOMM 2016*.

[33] Umesh Krishnaswamy, Rachee Singh, Nikolaj Bjørner, and Himanshu Raj. [n.d.]. Decentralized cloud wide-area network traffic engineering with BLASTSHIELD. In *NSDI 2022*.

[34] Jeffrey C Mogul, Rebecca Isaacs, and Brent Welch. [n.d.]. Thinking about availability in large service infrastructures. In *HotOS 2017*.

[35] Brandon Schlinker, Hyojeong Kim, Timothy Cui, Ethan Katz-Bassett, Harsha V. Madhyastha, Italo Cunha, James Quinn, Saif Hasan, Petr Lapukhov, and Hongyi Zeng. [n.d.]. Engineering Egress with Edge Fabric: Steering Oceans of Content to the World. In *SIGCOMM 2017*.

[36] Wes Davis. 2023. Netflix ends a three-year legal dispute over Squid Game traffic. https://theverge.com/2023/9/18/23879475/netflix-squid-game-sk-broadband-partnership

[37] Amogh Dhamdhere, David D Clark, Alexander Gamero-Garrido, Matthew Luckie, Ricky KP Mok, Gautam Akiwate, Kabir Gogia, Vaibhav Bajpai, Alex C Snoeren, and Kc Claffy. 2018. Inferring persistent interdomain congestion. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 1–15.

[38] Jinyang Li, Zhenyu Li, Ri Lu, Kai Xiao, Songlin Li, Jufeng Chen, Jingyu Yang, Chunli Zong, Aiyun Chen, Qinghua Wu, et al. [n.d.]. Livenet: a low-latency video transport network for large-scale live streaming. In *SIGCOMM 2022*.

[39] Yarin Perry, Felipe Vieira Frujeri, Chaim Hoch, Srikanth Kandula, Ishai Menache, Michael Schapira, and Aviv Tamar. [n.d.]. DOTE: Rethinking (Predictive) WAN Traffic Engineering. In *NSDI 2023*.

[40] Mark Jackson. 2020. Record Internet Traffic Surge Seen by UK ISPs on Tuesday. https://ispreview.co.uk/index.php/2020/11/record-internet-traffic-surge-seen-by-some-uk-isps-on-tuesday.html

[41] Martyn Landi. 2023. Return of original Fortnite map causes record traffic on Virgin Media O2 network. https://independent.co.uk/tech/fortnite-twitter-b2442476.html

[42] Scott Moritz. 2021. Internet Traffic Surge Triggers Massive Outage on East Coast. https://bloomberg.com/news/articles/2021-01-26/internet-outage-hits-broad-swath-of-eastern-u-s-customers

[43] Doug Madory. 2024. GP Leak Leads to Spike of Misdirected Traffic. https://kentik.com/analysis/BGP-Routing-Leak-Leads-to-Spike-of-Misdirected-Traffic

[44] Andrew Griffin. 2024. Facebook, Instagram, Messenger down: Meta platforms suddenly stop working in huge outage. https://independent.co.uk/tech/facebook-instagram-messenger-down-not-working-latest-b2507376.html

[45] Doug Madory. 2024. Outage Notice From Microsoft. https://twitter.com/DougMadory/status/1768286812894605517

[46] Matt Calder, Ashley Flavel, Ethan Katz-Bassett, Ratul Mahajan, and Jitendra Padhye. [n.d.]. Analyzing the Performance of an Anycast CDN. In *IMC 2015*.

[47] Thomas Koch, Ethan Katz-Bassett, John Heidemann, Matt Calder, Calvin Ardi, and Ke Li. [n.d.]. Anycast in Context: A Tale of Two Systems. In *SIGCOMM 2021*.

[48] Jiangchen Zhu, Kevin Vermeulen, Italo Cunha, Ethan Katz-Bassett, and Matt Calder. [n.d.]. The Best of Both Worlds: High Availability CDN Routing Without Compromising Control. In *IMC 2022*.

[49] Minyuan Zhou, Xiao Zhang, Shuai Hao, Xiaowei Yang, Jiaqi Zheng, Guihai Chen, and Wanchun Dou. [n.d.]. Regional IP Anycast: Deployments, Performance, and Potentials. In *SIGCOMM 2023*.

[50] Patrick Shuff. [n.d.]. Building a Billion User Load Balancer. https://usenix.org/conference/lisa16/conference-program/presentation/shuff

[51] Rupa Krishnan, Harsha V. Madhyastha, Sridhar Srinivasan, Sushant Jain, Arvind Krishnamurthy, Thomas Anderson, and Jie Gao. [n.d.]. Moving Beyond End-to-End Path Information to Optimize CDN Performance. In *IMC 2009*.

[52] Fangfei Chen, Ramesh K. Sitaraman, and Marcelo Torres. [n.d.]. End-User Mapping: Next Generation Request Routing for Content Delivery. In *SIGCOMM 2015*.

[53] Zhihao Li, Dave Levin, Neil Spring, and Bobby Bhattacharjee. [n.d.]. Internet Anycast: Performance, Problems, & Potential. In *SIGCOMM 2018*.

[54] Ann Bednarz. 2023. Global Microsoft cloud-service outage traced to rapid BGP router updates. https://networkworld.com/article/971873/global-microsoft-cloud-service-outage-traced-to-rapid-bgp-router-updates.html

[55] Santosh Janardhan. 2021. More details about the October 4 outage. https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/

[56] Ben Cartwright-Cox. 2023. Grave flaws in BGP Error handling. https://edgecast.medium.com/the-cdn-edge-brings-compute-closer-to-where-it-is-needed-most-d4a3f4107b11

[57] Maxime Piraux, Louis Navarre, Nicolas Rybowski, Olivier Bonaventure, and Benoit Donnet. [n.d.]. Revealing the evolution of a cloud provider through its network weather map. In *IMC 2022*.

[58] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, et al. [n.d.]. Jupiter evolving: transforming google's datacenter network via optical circuit switches and software-defined networking. In *SIGCOMM 2022*.

[59] Rachee Singh, Sharad Agarwal, Matt Calder, and Paramvir Bahl. [n.d.]. Cost-Effective Cloud Edge Traffic Engineering with Cascara. In *NSDI 2021*.

[60] NANOG. 2022. Panel: Buying and Selling IPv4 Addresses. https://youtube.com/watch?v=8FlTJEct9_s

[61] Geoff Huston. 2025. How we Measure: IPv6. https://www.potaroo.net/ispcol/2025-11/measure-ipv6.html

[62] Geoff Huston. 2025. IP Addresses through 2025. https://www.potaroo.net/ispcol/2026-01/addr2025.html

[63] 2026. Google IPv6 Statistics. https://www.google.com/intl/en/ipv6/statistics.html

[64] RIPE NCC Staff. 2023. RIS Live. (2023). https://ris-live.ripe.net

[65] Ticiane Takami. 2021. Project Myriagon: Cloudflare Passes 10,000 Connected Networks. https://blog.cloudflare.com/10000-networks-and-beyond/

[66] Pavlos Sermpezis and Vasileios Kotronis. 2019. Inferring catchment in internet routing. (2019).

[67] Microsoft. 2023. Microsoft Datacenters. https://datacenters.microsoft.com/

[68] Todd Arnold, Jia He, Weifan Jiang, Matt Calder, Italo Cunha, Vasileios Giotsas, and Ethan Katz-Bassett. [n.d.]. Cloud Provider Connectivity in the Flat Internet. In *IMC 2020*.

[69] Costin Raiciu, Christoph Paasch, Sebastien Barre, Alan Ford, Michio Honda, Fabien Duchene, Olivier Bonaventure, and Mark Handley. [n.d.]. How Hard Can It Be? Designing and Implementing a Deployable Multipath TCP. In *NSDI 2012*.

[70] Quentin De Coninck and Olivier Bonaventure. [n.d.]. Multipath QUIC: Design and Evaluation. In *CoNEXT 2017*.

[71] Quentin De Coninck and Olivier Bonaventure. 2021. Multiflow QUIC: A generic multipath transport protocol. *IEEE Communications Magazine* (2021).

[72] Patrick Sattler, Juliane Aulbach, Johannes Zirngibl, and Georg Carle. [n.d.]. Towards a Tectonic Traffic Shift? Investigating Apple's New Relay Network. In *IMC 2022*.

[73] Matt Calder, Ryan Gao, Manuel Schröder, Ryan Stewart, Jitendra Padhye, Ratul Mahajan, Ganesh Ananthanarayanan, and Ethan Katz-Bassett. [n.d.]. Odin: Microsoft's Scalable Fault-Tolerant CDN Measurement System. In *NSDI 2018*.

[74] Wouter B. De Vries, Ricardo de O. Schmidt, Wes Hardaker, John Heidemann, Pieter-Tjerk de Boer, and Aiko Pras. [n.d.]. Broad and Load-Aware Anycast Mapping with Verfploeter. In *IMC 2017*.

[75] Palak Goenka, Kyriakos Zarifis, Arpit Gupta, and Matt Calder. 2022. Towards client-side active measurements without application control. *SIGCOMM CCR 2022* (2022).

[76] [n.d.]. Shannon Entropy. https://en.wikipedia.org/wiki/Entropy_(information_theory)

[77] Sophie Langer. 2021. Approximating smooth functions by deep neural networks with sigmoid activation function. *Journal of Multivariate Analysis 2021* (2021).

[78] Yoav Shechtman, Amir Beck, and Yonina C Eldar. 2014. GESPAR: Efficient phase retrieval of sparse signals. *IEEE transactions on signal processing* (2014).

[79] Yurii Nesterov et al. 2018. *Lectures on convex optimization*. Vol. 137. Springer.

[80] Haochuan Li, Jian Qian, Yi Tian, Alexander Rakhlin, and Ali Jadbabaie. 2024. Convex and non-convex optimization under generalized smoothness. *Advances in Neural Information Processing Systems* 36 (2024).

[81] SciPy developers. 2024. SciPy linprog (HiGHS). https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html

[82] Hongqiang Harry Liu, Raajay Viswanathan, Matt Calder, Aditya Akella, Ratul Mahajan, Jitendra Padhye, and Ming Zhang. [n.d.]. Efficiently Delivering Online Services over Integrated Infrastructure. In *NSDI 2016*.

[83] Kok-Kiong Yap, Murtaza Motiwala, Jeremy Rahe, Steve Padgett, Matthew Holliman, Gary Baldus, Marcus Hines, Taeeun Kim, Ashok Narayanan, Ankur Jain, et al. [n.d.]. Taking the Edge off with Espresso: Scale, Reliability and Programmability for Global Internet Peering. In *SIGCOMM 2017*.

[84] Zheng Zhang, Ming Zhang, Albert G. Greenberg, Y. Charlie Hu, Ratul Mahajan, and Blaine Christian. [n.d.]. Optimizing Cost and Performance in Online Service Provider Networks. In *NSDI 2010*.

[85] Raul Landa, Lorenzo Saino, Lennert Buytenhek, and João Taveira Araújo. [n.d.]. Staying Alive: Connection Path Reselection at the Edge. In *NSDI 2021*.

[86] ASM Rizvi, Leandro Bertholdo, João Ceron, and John Heidemann. [n.d.]. Anycast Agility: Network Playbooks to Fight DDoS. In *USENIX Security Symposium 2022*.

[87] Jiangchen Zhu, Tom Koch, Ilgar Mammadov, Shuyue Yu, Kevin Vermeulen, Matt Calder, Italo Cunha, and Ethan Katz-Bassett. [n.d.]. The New (Pareto) Frontier of Cloud Routing: High Availability, Precise Control, or Configuration Stability-Choose Two. *CoNEXT 2025*.

[88] Nikolaos Chatzis, Georgios Smaragdakis, Anja Feldmann, and Walter Willinger. 2013. There is more to IXPs than meets the eye. *SIGCOMM Computer Communication Review* (2013).

[89] Geoff Huston. 2014. How Big is that Network? labs.apnic.net/?p=526

[90] Loqman Salamatian, Calvin Ardi, Vasileios Giotsas, Matt Calder, Ethan Katz-Bassett, and Todd Arnold. [n.d.]. What's in the Dataset? Unboxing the APNIC per AS User Population Dataset. In *IMC 2024*.

[91] RIPE NCC Staff. 2015. RIPE Atlas: A Global Internet Measurement Network. *Internet Protocol Journal* (2015).

[92] Danilo Giordano, Danilo Cicalese, Alessandro Finamore, Marco Mellia, Maurizio Munafò, Diana Zeaiter Joumblatt, and Dario Rossi. [n.d.]. A First Characterization of Anycast Traffic from Passive Traces. In *TMA 2016*.

[93] Danilo Cicalese, Jordan Augé, Diana Joumblatt, Timur Friedman, and Dario Rossi. [n.d.]. Characterizing IPv4 Anycast Adoption and Deployment. In *CoNEXT 2015*.

[94] Verizon. 2020. Edgecast. https://verizondigitalmedia.com/media-platform/delivery/network/

[95] Matthew Prince. 2013. Load Balancing without Load Balancers. https://blog.cloudflare.com/cloudflares-architecture-eliminating-single-p/

[96] Jing'an Xue, Weizhen Dang, Haibo Wang, Jilong Wang, and Hui Wang. [n.d.]. Evaluating Performance and Inefficient Routing of an Anycast CDN. In *International Symposium on Quality of Service 2019*.

[97] Ye Wang, Hao Wang, Ajay Mahimkar, Richard Alimi, Yin Zhang, Lili Qiu, and Yang Richard Yang. [n.d.]. R3: resilient routing reconfiguration. In *SIGCOMM 2010*.

[98] Hongqiang Harry Liu, Srikanth Kandula, Ratul Mahajan, Ming Zhang, and David Gelernter. [n.d.]. Traffic engineering with forward fault correction. In *SIGCOMM 2014*.

[99] Tobias Flach, Nandita Dukkipati, Andreas Terzis, Barath Raghavan, Neal Cardwell, Yuchung Cheng, Ankur Jain, Shuai Hao, Ethan Katz-Bassett, and Ramesh Govindan. [n.d.]. Reducing web latency: the virtue of gentle aggression. In *SIGCOMM 2013*.

[100] David Wetherall, Abdul Kabbani, Van Jacobson, Jim Winget, Yuchung Cheng, Charles B Morrey III, Uma Moravapalle, Phillipa Gill, Steven Knight, and Amin Vahdat. [n.d.]. Improving Network Availability with Protective ReRoute. In *SIGCOMM 2023*.

[101] Electicity Maps. 2026. Electicity Maps. https://www.electricitymaps.com/data

[102] Sawsan El-Zahr, William Nathan, and Noa Zilberman. [n.d.]. Carbon-Intelligent Content Scheduling in CDNs. In *ANRW 2025*.

[103] Minyuan Zhou, Yuning Chen, Jiaqi Zheng, Yifei Xu, Pan Hu, Yongping Tang, Wendong Yin, Jie Lin, Qingyan Yu, Yuanchao Su, Guihai Chen, Wanchun Dou, Songwu Lu, and Wan Du. [n.d.]. AnyPro: Preference-Preserving Anycast Optimization based on Strategic AS-Path Prepending. In *NSDI 2026*.

[104] Vytautas Valancius, Bharath Ravi, Nick Feamster, and Alex C Snoeren. [n.d.]. Quantifying the Benefits of Joint Content and Network Routing. In *SIGMETRICS 2013*.

[105] Henry Birge-Lee, Sophia Yoo, Benjamin Herber, Jennifer Rexford, and Maria Apostolaki. [n.d.]. {TANGO}: Secure Collaborative Route Control across the Public Internet. In *NSDI 2024*.

[106] Minyuan Zhou, Jiaqi Zheng, Congying Wang, Guihai Chen, and Wanchun Dou. [n.d.]. AnyTuner: Optimizing IP Anycast Performance via Strategic BGP Routing Policy. In *ICNP 2025*.

[107] Subspace. 2022. Optimize Your Network on Subspace. https://subspace.com/solutions/reduce-internet-latency

[108] INAP. 2022. INAP Network Connectivity. https://inap.com/network/

[109] Akamai. 2022. Akamai Secure Access Service Edge. https://akamai.com/resources/akamai-secure-access-service-edge-sase

[110] Cloudflare. 2022. Argo Smart Routing. https://cloudflare.com/products/argo-smart-routing/

[111] John P Eason, Xueqi He, Richard Cziva, Max Noormohammadpour, Srivatsan Balasubramanian, Satyajeet Singh Ahuja, and Biao Lu. [n.d.]. Hose-based cross-layer backbone network design with Benders decomposition. In *SIGCOMM 2023*.

[112] Yiyang Chang, Chuan Jiang, Ashish Chandra, Sanjay Rao, and Mohit Tawarmalani. 2019. Lancet: Better network resilience by designing for pruned failure sets. (2019).

[113] Chuan Jiang, Sanjay Rao, and Mohit Tawarmalani. [n.d.]. PCF: provably resilient flexible routing. In *SIGCOMM 2020*.

[114] IETF. 2005. Fast Reroute Extensions to RSVP-TE for LSP Tunnels. https://datatracker.ietf.org/doc/html/rfc4090

[115] IETF. 2006. BGP/MPLS IP Virtual Private Networks (VPNs). https://datatracker.ietf.org/doc/html/rfc4364

[116] IETF. 2000. A Framework for Integrated Services Operation over Diffserv Networks. https://datatracker.ietf.org/doc/html/rfc2998

[117] IETF. 2023. Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture. https://datatracker.ietf.org/doc/rfc9330/

[118] Wen Xu and Jennifer Rexford. [n.d.]. MIRO: Multi-Path Interdomain Routing. In *SIGCOMM 2006*.

[119] Sharad Agarwal, Chen-Nee Chuah, Supratik Bhattacharyya, and Christophe Diot. [n.d.]. Impact of BGP Dynamics on Router CPU Utilization. In *PAM 2004*.

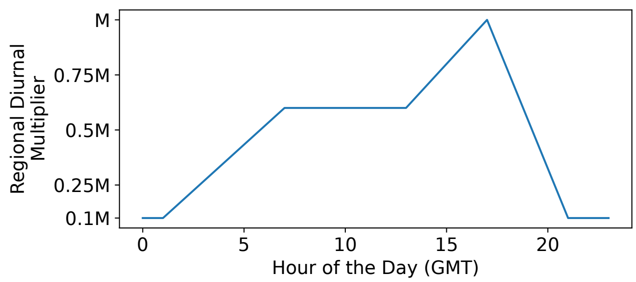


Figure 8: The diurnal traffic shape we apply in Section 5.3.2, taken from prior work [20].

A Methodology Clarifications

A.1 Definition of Terms

Table 2 includes a table of common terms and expressions used in Section 3, for ease of reference.

A.2 One Iteration

Putting the pieces together, one iteration of SCULPTOR proceeds as follows (Figure 3).

1. Threshold the relaxed configuration $\tilde{A}$ at 0.5 to obtain the current advertisement *mathitA* (§3.5.2).

2. If the model's entropy is high, or enough iterations have passed since the last measurement, advertise A via BGP and observe the ingress each UG uses for each prefix compared to what was available to update the routing model (§3.4.2).

3. Draw K routing functions R from the current model, one ingress per UG and prefix (§3.4.1).

4. For a random subset of entries (p,l) of $\tilde{A}$, solve the traffic allocation with $A(p,l)$ set to 1 and to 0 under each sampled R (§3.6); the averaged differences, passed through a sigmoid, are the gradient entries.

5. Take a Nesterov step on $\tilde{A}$.

6. Repeat until the measurement budget is exhausted or the objective stops improving (**??**).

B Considerations for Real-World Deployments

In Section 4 we discuss how we prototype SCULPTOR. Here we give a high-level overview of how a real cloud can practically, safely implement SCULPTOR. The measurement, advertisement, and learning process described in Section 3 is meant to be carried out before any production traffic is shifted to the prefixes SCULPTOR is configuring, as in prior work [46, 16, 1].

SCULPTOR requires latency measurements to each ingress. To obtain these, the Service Provider should direct clients to measure latency towards a test prefix using a measurement system similar to those found in the literature [73, 52, 74, 75]. Note that this process *does not involve production traffic*. The Service Provider should then advertise the test prefix to one ingress at a time and record client latencies to the ingress.

With these initial measurements, Service Providers can initialize the learning process described in Section 3.4.2 using test prefixes. Prior work also shows that BGP routing changes minimally disrupt routers [119], so SCULPTOR's learning process using test prefixes does not affect production traffic.

After convergence, SCULPTOR can slowly shift production traffic over to the optimal learned configuration. Note that, since most traffic is already routed along a good path [51, 46, 16, 1], SCULPTOR may only need to shift a small fraction of traffic or do so infrequently [6], mitigating the blast radius of configuration changes. The learning process is very infrequent, as prior work found that advertisements do not need to be updated even after a month [1].

C Diurnal Traffic Shape

In Figure 8, we show the shape of our diurnal curve taken from observed diurnal traffic patterns at a Service Provider [20]. We use this diurnal traffic pattern to evaluate how well balances traffic across sites (§5.3.2). Specifically, we linearly interpolate 6 points on the purple curve (Edge Node 1) in Figure 1 of that paper, capturing the essential peaks and troughs.

Variable	
A	Advertisement configuration over peers and prefixes
w	Traffic allocation to routes
R	Routing function, which maps advertisements to routes. Unknown at first but updated according to measurements
p	Prefix, one of many in a prefix budget (limited due to expenses)
l	Ingress link, Service Providers often have thousands
$c(l)$	Link l capacity
`UG`	User group, users that route similarly. *e.g.,*/24's or metros
$w(p, \texttt{UG})$	Amount of traffic from users in `UG` we direct towards prefix p
G	Objective function, *e.g.,* average latency
$v(\texttt{UG})$	Traffic volume for users `UG`
F_l	Failure on link l
β	Tradeoff parameter between minimizing latency and maximum link utilization
M	Maximum link utilization
α_l	Importance parameter for failure on link l, *e.g.,* if one failure is more likely than the other
Expression	
$l = R(A)(p, \texttt{UG})$	Route (ingress link l) resulting from A, for `UG` towards p
$\mathcal{L}(\texttt{UG}, l)$	Latency for `UG` across l
$G(R(A), w)$	Objective function value under A and w

Table 2: Table of common terms and expressions.